

AR Broken Hoops

Game Design Document

Authors

Ibrahim Labib, Abdelrahman El Sherbiny, and Youssef El Helw

Reviewed By

Dr. Osama ElSharkawy, and Dr. Adam Hassan

Created

19/5/2026

Institution

ESLSCA University

1. Game Overview

1.1 Game Title

Broken Hoops

1.2 Genre

Augmented Reality Sports / Casual Arcade Game

Broken Hoops is a casual AR basketball game where players place a virtual basketball hoop in their real-world environment and shoot basketballs using touch, drag, and flick-based controls.

1.3 Platform

Primary Platform: Android Mobile

Development Engine: Unity 6 / Unity 6000.3.8f1

AR Framework: AR Foundation

Testing Environment: XR Simulation

Input Method: Touchscreen tap, drag, and flick

The game is designed for Android mobile devices and can also be tested inside the Unity Editor using XR Simulation.

2. High Concept

Broken Hoops allows players to turn their real environment into a virtual basketball court. Players place a basketball hoop using either markerless AR plane detection or marker-based image tracking, then shoot a virtual basketball using flick-based controls.

The game expands beyond a basic AR object placement demo by including scoring, timers, multiple game modes, physics-based throwing, customization, audio, UI screens, and end-game feedback.

3. Target Audience

The game is designed for casual mobile players who enjoy simple, replayable sports games and interactive AR experiences.

It is also suitable as an academic AR final project because it demonstrates multiple AR development concepts, including:

- Markerless plane detection
 - Marker-based image tracking
 - Physics-based interaction
 - Game mode management
 - UI navigation
 - Scoring and feedback systems
-

4. Design Goals

The main design goals of Broken Hoops are:

1. Create a playable AR basketball experience rather than a simple AR placement demo.
 2. Support both markerless and marker-based AR interaction.
 3. Provide simple and intuitive mobile controls.
 4. Include multiple game modes for replayability.
 5. Use physics-based throwing to make gameplay feel interactive.
 6. Give players clear feedback through scoring, timers, music, and end-game ratings.
 7. Demonstrate a complete Unity AR project structure suitable for final project evaluation.
-

5. Core Gameplay Loop

The gameplay loop is designed to be simple, repeatable, and easy to understand.

1. Player opens the game.
2. Player chooses a game mode and trial settings.
3. Player enters the AR game scene.
4. Player places the hoop using markerless placement or marker tracking.
5. Player confirms hoop placement.
6. A basketball spawns in front of the camera.
7. Player throws the ball using drag and flick controls.
8. The ball either scores or misses.

9. The score updates or the ball respawns.
 10. Gameplay continues until the selected mode ends.
 11. The end-game screen displays the final score and rating image.
-

6. Gameplay Objective

The main objective is to score baskets by throwing the ball into the hoop.

Depending on the selected mode, the player may:

- Play freely without a timer
 - Score as many points as possible within a time limit
 - Deal with randomized ball flight behavior
 - Attempt creative or difficult trick shots
-

7. Game Modes

7.1 Sandbox Mode

Sandbox Mode allows the player to play freely without a timer.

Purpose:

- Practice shooting
- Explore AR placement
- Play without pressure

7.2 Time Trial Mode

Time Trial Mode challenges the player to score as many points as possible within a selected time limit.

Available time options:

- 10 seconds
- 30 seconds
- 1 minute
- 2 minutes

- 3 minutes
- 5 minutes

Default Settings:

- Spawn Mode: Markerless
- Game Mode: Time Trial
- Time Limit: 1 minute

When the timer ends, the game displays the final score and an end-game rating image.

7.3 Flight Style Mode

Flight Style Mode is a more difficult version of Time Trial. In this mode, the ball's trajectory is randomized after being thrown.

Possible randomized behaviors include:

- Overshooting
- Undershooting
- Curving left
- Curving right
- Launching with unexpected strength

This mode creates a more challenging and unpredictable gameplay experience.

Flight Style Mode also uses a separate music playlist from the standard game modes.

7.4 Trick Shot Mode

Trick Shot Mode focuses on rewarding creative and difficult shots.

Possible trick shot challenges include:

- Score from far away
- Score with a high arc
- Score after a bounce
- Score quickly after the ball spawns
- Score multiple baskets in a row

This mode increases replay value by giving the player more creative objectives.

8. Core Mechanics

8.1 Hoop Placement

The game supports two AR placement methods.

Markerless Placement

The player scans the environment and taps on a detected floor plane. The hoop is then spawned at the selected location.

Main systems used:

- AR Plane Manager
- AR Raycast Manager
- ARInputHandler
- ARPlaneHoopPlacement

Marker-Based Placement

The player points the camera at a basketball marker image. When the marker is detected, the hoop appears attached to or positioned above the marker.

Main systems used:

- AR Tracked Image Manager
 - XR Reference Image Library
 - SimulatedTrackedImage for Unity Editor testing
 - ARImageTracker
-

8.2 Ball Spawning

The basketball spawns in front of the player using an anchor positioned under the AR camera.

Main object used:

- BallSocketAnchor

This ensures that the ball appears consistently in the player's view and is easy to interact with.

8.3 Throwing Mechanic

The player throws the ball using touch or mouse input.

In the Unity Editor, throwing is tested with mouse input. On Android, the player uses touch drag and flick input.

Main components used:

- Rigidbody
- Sphere Collider
- BasketballController
- BallGrabThrowInput

Throw strength can be adjusted using sensitivity settings to make the experience feel fair and responsive.

8.4 Scoring

The hoop contains a score trigger positioned inside the rim. When the basketball passes through the trigger, the game counts the shot as successful.

When a score occurs:

- The score increases
 - The ball is disabled or destroyed
 - A new ball respawns
 - The UI score display updates
-

8.5 Miss Handling

If the ball misses, it can be automatically destroyed and respawned after a short delay.

This prevents the player from needing to physically chase the ball around the AR space.

The game may also include a player radius system that removes balls when they travel too far from the player.

9. Scoring and End-Game Rating

At the end of a timed game, the final score determines which rating image is displayed.

Final Score	Rating
--------------------	---------------

0 points	Bad
----------	-----

1–5 points	OK
------------	----

6–10 points	Good
-------------	------

11–15 points	Great
--------------	-------

16+ points	Awesome
------------	---------

The end-game screen contains five possible rating images:

- FlightImageBad
- FlightImageOK
- FlightImageGood
- FlightImageGreat
- FlightImageAwesome

Only one image is displayed based on the final score.

10. User Interface Design

The game uses separate UI panels for the main menu and gameplay scene to keep navigation clean and organized.

10.1 Main Menu UI

Suggested hierarchy:

MainMenuCanvas

└─ DefaultPanel

└─ ModePanel

└─ TrialsPanel

└─ SettingsPanel

└─ AboutPanel

Default Panel

Contains the main navigation buttons:

- Mode
- Trials
- Settings
- Start
- Exit

The Start button begins the game using the default settings.

Mode Panel

Allows the player to choose between:

- Sandbox
- Time Trial
- Flight Style
- Trick Shot
- Markerless placement
- Marker-based placement

Trials Panel

Contains the time dropdown list used to select the Time Trial duration.

Settings Panel

Allows the player to adjust game settings such as volume.

About Panel

Displays project information, including the game title, creator details, and basic project description.

10.2 Gameplay UI

Suggested hierarchy:

GameplayCanvas

├── HUDPanel

├── SetupPanel

├── CountdownPanel

├── EndGamePanel

├── BasketballSelectionPanel

├── BackboardColorPanel

└── SettingsPanel

HUD Panel

Displays:

- Current score
- Timer
- Current game mode

Setup Panel

Displays placement instructions and the confirm placement button.

Countdown Panel

Displays the countdown before gameplay begins.

End Game Panel

Displays:

- Game over message
- Final score
- Score rating image
- Retry button
- Main menu button

Basketball Selection Panel

Allows the player to switch between different basketball appearances.

Backboard Color Panel

Allows the player to change the hoop backboard color.

Settings Panel

Allows the player to adjust sound, sensitivity, socket settings, and return to the main menu.

11. Audio Design

The game includes a jukebox system that plays random songs.

There are two separate playlists:

Standard Playlist

Used for:

- Sandbox Mode
- Time Trial Mode
- Trick Shot Mode

Flight Style Playlist

Used only for:

- Flight Style Mode

The jukebox automatically plays a random song when the scene starts. When a song ends, another random song from the correct playlist plays automatically.

The player can also press the Jukebox button to manually play a new random song.

12. Visual Design

Broken Hoops uses a basketball-themed visual style with bold UI elements, sports-style presentation, and clear feedback.

The main menu includes a rotating basketball icon to make the interface feel more dynamic.

The AR scene includes:

- Basketball hoop
 - Basketball
 - Backboard
 - Rim
 - Score trigger
 - AR plane visuals
 - Marker image
 - End-game rating images
-

13. Prefab Design

13.1 Basketball Prefab

The basketball prefab begins as an imported 3D model and is enhanced with Unity components.

Recommended components:

- Mesh Filter

- Mesh Renderer
- Rigidbody
- Sphere Collider
- XR Grab Interactable
- BasketballController
- BallGrabThrowInput
- Trail Renderer, optional
- Audio Source, optional

The ball must use physics so that it can be thrown naturally.

13.2 Hoop Prefab

The hoop prefab contains:

- Backboard
- Rim
- Optional net
- ScoreTrigger
- HoopController

The score trigger should be positioned inside the rim and should use a trigger collider.

13.3 Marker Prefab / Simulated Image

For Unity Editor testing, XR Simulation uses a simulated tracked image.

The simulated marker must match the image stored in the Reference Image Library.

14. Technical Design

14.1 Unity Systems Used

The project uses the following Unity systems and packages:

- Unity 6 / Unity 6000.3.8f1

- AR Foundation
 - XR Simulation
 - Android SDK
 - ARCore XR Plugin
 - XR Interaction Toolkit
 - TextMeshPro
 - Unity Physics
 - Unity UI
-

14.2 AR Scene Components

The XR Origin object contains:

- AR Plane Manager
- AR Raycast Manager
- AR Tracked Image Manager
- Input Action Manager

The AR Session and XR Origin transforms should remain reset:

Position: 0, 0, 0

Rotation: 0, 0, 0

Scale: 1, 1, 1

This is important because incorrect AR origin transforms can cause tracking and placement alignment issues.

14.3 Reference Image Library

The Reference Image Library stores the marker images that AR Foundation can detect.

Example marker setup:

Name: Basketball

Texture: Basketball marker texture

Physical Size: 1m x 1m for simulation

For Android testing, the physical size should match the real printed marker size.

15. Main Scripts

15.1 AR Scripts

- ARInputHandler.cs
- ARPlaneHoopPlacement.cs
- ARImageTracker.cs

15.2 Game Management Scripts

- GameSessionSettings.cs
- BrokenHoopsGameManager.cs
- HoopManager.cs
- BallSpawnManager.cs
- GameplayUIManager.cs
- MainMenuManager.cs

15.3 Ball Scripts

- BasketballController.cs
- BallGrabThrowInput.cs

15.4 Hoop Scripts

- HoopController.cs
- ScoreTrigger.cs

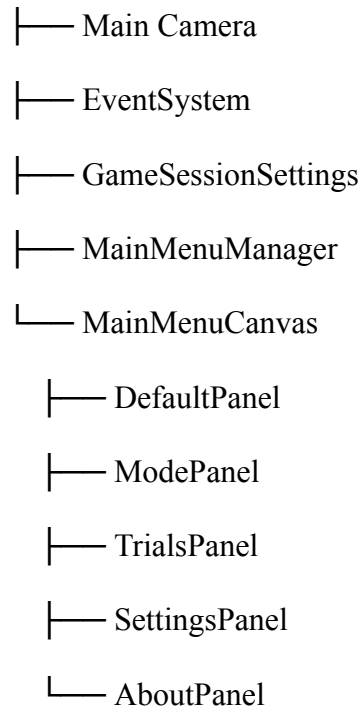
15.5 Extra Feature Scripts

- JukeboxManager.cs
- TrickShotChallengeManager.cs
- PlayerRadiusMonitor.cs

16. Scene Structure

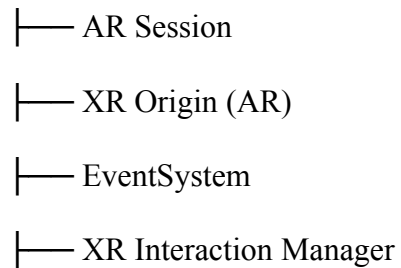
16.1 Main Menu Scene

MainMenu



16.2 AR Game Scene

AR_Game



- |— GameSessionSettings
- |— AR Input Handler
- |— ARManagers
- |— GameManagers
- |— GameplayCanvas
- |— Directional Light

17. Controls

17.1 Unity Editor Controls

Action	Input
Move in XR Simulation	Right Mouse + WASD
Tap UI or detected plane	Mouse Click
Throw basketball	Mouse Drag / Release

17.2 Android Controls

Action	Input
Select UI	Touch Tap

Place hoop Touch Tap

Throw basketball Touch Drag / Flick

Detect marker Camera View

18. Win and Lose Conditions

Broken Hoops does not use a traditional lose condition.

Instead, the player is evaluated based on score.

In timed modes, the game ends when the timer reaches zero. The final score determines the end-game rating:

- Bad
 - OK
 - Good
 - Great
 - Awesome
-

19. Replayability

Replayability is created through:

- Beating previous scores
- Playing different time trials
- Randomized Flight Style gameplay
- Basketball customization
- Backboard color customization
- Trick Shot challenges
- Different AR placement methods

20. Risks and Challenges

20.1 AR Tracking Reliability

Marker-based tracking depends on:

- Good marker image quality
- Correct physical marker size
- Correct Reference Image Library setup
- Correct XR Origin transform
- Good lighting
- Clear camera visibility

20.2 Scene Reload Issues

AR scenes can be sensitive when switching between the Main Menu and AR_Game scenes.

To reduce issues, AR Session and XR Origin objects should remain scene-local unless intentionally made persistent.

20.3 Physics Tuning

The following values must be balanced carefully:

- Throw strength
- Ball mass
- Ball drag
- Flick sensitivity
- Respawn delay

Poor tuning can make shots feel either too easy or too difficult.

20.4 UI Scaling

The UI must use proper Canvas Scaler settings so it appears correctly across different mobile screen sizes.

21. Testing Plan

21.1 Placement Testing

Test the following:

- Markerless placement on detected planes
- Marker-based placement with a simulated marker
- Marker-based placement on Android with a printed marker

21.2 Gameplay Testing

Test the following:

- Scoring
- Ball respawn
- Timer ending
- Retry button
- Main menu return
- All game modes

21.3 UI Testing

Test the following:

- Main menu navigation
- Mode selection
- Time dropdown selection
- Settings panel
- About panel
- End-game images

21.4 Audio Testing

Test the following:

- Standard playlist
- Flight Style playlist
- Automatic startup music
- Random next-song behavior
- Manual jukebox button behavior

22. Future Improvements

Possible future improvements include:

- Leaderboard system
- Better trick shot detection
- More hoop styles
- More basketball skins
- Particle effects for scoring
- Sound effects for ball bounce and scoring
- Improved Android touch controls
- More polished menu animations
- Multiplayer score challenges

23. Conclusion

Broken Hoops is an augmented reality basketball game that combines AR placement, physics-based throwing, scoring, multiple game modes, customization, audio, and UI systems.

The project demonstrates both markerless plane detection and marker-based image tracking while also providing a complete arcade-style gameplay loop.

By including scoring, timers, game modes, end-game feedback, and customization, Broken Hoops goes beyond a basic AR object placement demo and becomes a more complete interactive AR game experience.